Evaluating Claude Sonnet 4.5 for Legacy Code Understanding and Application Development

Justin Archibald, Rafael Rojero, Jackson Seales, Aaron Elofson, Dakota Sicher
Department of Computer Science
University of Colorado Colorado Springs
Email: {justin.archibald, rrojer, jseales, aelofson, dsicher}@uccs.edu

***Abstract*—This research evaluates how effectively Claude Sonnet 4.5 can understand, modify, and extend an existing application codebase. Unlike prior work that focuses on AI as a general productivity tool, our study investigates the model's performance specifically within a legacy software environment. We measure regression rates, mutation test accuracy, reviewer quality scores, maintainability, readability, and the number of manual corrections required per task. Our findings highlight Claude's strengths in rapid feature development, CI/CD integration, and secure coding patterns, while revealing weaknesses in testing stability, linting, configuration handling, and working within complex pre-existing codebases. The goal of this work is to provide measurable criteria and empirical analysis for evaluating AI-assisted development in real software projects.**

I. Introduction

AI-powered development tools are becoming widely used across the software industry and are improving with incredible speed. One of the newest, Claude Sonnet 4.5 was release on September 29, 2025, and has been hailed by Anthropic as the "best coding model in the world." Reliability on similar tools from OpenAI, Google, and others has exploded in the last few years, which has motivated research into the efficacy of these tools.

While many studies evaluate their productivity impact, little research investigates whether an AI model can reliably work within an existing code-base especially one containing legacy logic, incomplete documentation, or fragmented structure. For a significant portion of the Software Engineering world, working on "legacy" code is the norm. "Legacy" code can be defined as any code base that is poorly documented, unorganized, or difficult to understand. Engineers who used to work with and maybe understand the code are no longer available for comment. This means legacy code could be from 20 years ago, or last week.

While Claude may be able to produce code for new projects quickly, even following good practices for the most part, we want to explore Claude's capabilities when being introduced to existing code, with poor documentation, that may not be well structured. Additionally we want to see how Claude's capabilities to improve this code in all aspects. These improvements include standard implementation of features, testing, and documentation. The project we used to test Claude's ability provides plenty of opportunists for Claude to have to characterize poorly implemented features. The code-base was written by undergraduate students and small amount of time. This lead to documentation that wasn't entirely helpful and a testing suite that lacked coverage and failed to pass when handed off to Claude. Claude needed to somehow create a baseline understanding for this project and improve upon testing that was already there to be able to gauge success in future development.

Due to Claude Sonnet 4.5 having no real long term memory, every time it starts a new session it acts as tho it's working on "legacy" code. To create a novel environment to study Claude and it's ability to adapt existing code on an existing project, we will be disallowing ourselves to make and edits to code or documentation outside of LLM development focused files such as feature.md or agents.md files. Without being able to create code or documentation ourselves we have to see how Claude handles the changes it is making to the app, and how it create documents or comments on how to explain what it did in the past for future versions of itself to understand and adapt. This gives ample room to view Claude's ability to create future proof and manageable code, where it's strong points are and where it has room to improve.

A. Contribution

This paper makes three main contributions:

1) We define new metrics for evaluating AI performance on legacy applications, including debugging effectiveness, code quality, maintainability, readability, regression impact, and standalone-code accuracy.
2) We evaluate Claude Sonnet 4.5 using these metrics across multiple development tasks involving modification and extension of an existing application.
3) We analyze strengths, weaknesses, and recurring failure modes in Claude's development behavior to guide future research and practitioner usage.

II. Related Work

Prior literature primarily focuses on AI as a productivity assistant or code-generation tool. The 2025 DORA report highlights widespread adoption of generative AI among developers. Other work examines developer trust, job security, and productivity outcomes from AI coding assistants. However, existing studies do not address how well AI models comprehend and extend pre-existing codebases, nor do they define metrics for evaluating AI performance in legacy environments. Our study addresses this gap.

1) Agent-Computer interfaces enable Automated Software Engineering: Yang etal in NeurIPS 2024 used various language model agents to autonomously fix broken code by navigating, editing, and testing code with various interfaces [1]. Yang etal used their agents on several coding languages across Java, Python and Javascript [1]. Their work emphasizes automated issue resolution evaluated on repair benchmarks SWE-bench with a 12.47% pass rate & HumanEvalFix 88% pass rate [1]. The team observed that agents had difficulty with linting, failing slowly with fewer tokens used for successful implementations than unsuccessful ones, and often its test failures implemented the code incorrectly as we see later these issues were not unique to them [1].

2) LLM-Based Multi-Agent Systems for Software Engineering: Other research has explored the concept of integrating LLMs into software engineering through multi-agent systems that distribute development tasks across specialized roles [2]. These suggested frameworks organize agents into roles such as Product Manager, Architect, and QA Engineer, with each agent responsible for specific SDLC phases including requirements analysis, design, implementation, and testing. The system emphasizes workflows where multiple agents engage in iterative feedback loops. One source of research on the multi-agent system focuses primarily on Agile methodology, identifying existing research gaps and constructing a vision on the future of autonomous systems while estimating some key metrics from simulated environments [3]. This research aims at the SDLC for fresh software, as well as agents specialized for their particular tasks as opposed to our exploration of a single agent model for legacy code, where complications may arise with documentation, dependencies, and outdated technologies.

3) Code Quality and Security in LLM-Generated Code: Tosi [4] conducted an empirical evaluation comparing three LLMs (GPT-3.5, GPT-4, and Google Bard) on complex coding problems from university programming exams. The study employed a two-fold evaluation approach: functional correctness testing through custom test suites and quality assessment metrics including Lines of Code, Cyclomatic Complexity, Cognitive Complexity, and Code Smells [4]. Results demonstrated that while all three models could solve the problems with human supervision, they required constant expert intervention to produce functional, quality code. Notably, when presented with problems that had available online solutions (e.g., from LeetCode), all three models returned identical solutions, suggesting reliance on training data rather than independent problem-solving [4]. The study emphasized that LLMs currently need support from human experts throughout the development process and highlighted the importance of clear software requirements and well-defined coding problems [4].

4) Security Vulnerabilities in AI-Generated Code: Taeb et al. [5] investigated the security implications of generating code with AI tools by analyzing multiple agents across multiple vulnerability categories. The research employed a pipeline between the agents, each serving a different purpose and being prompted with a different part of the process [5]. Results revealed that one model was best at generating accurate and secure code, while another excelled at explaining vulnerabilities and analyzing code [5]. The study categorized user interactions into modes separated by the knowledge the programmers have on their objectives, either knowledgeable with intent to expedite or lacking knowledge and asking the model how to proceed from scratch. Notably, when some models generated code and were subsequently asked to review it, the model indicated no vulnerabilities were present, suggesting limitations in self-assessment capabilities [5].

III. Preliminaries

Legacy codebases often contain structural inconsistencies, undocumented patterns, outdated dependencies, and partial test coverage [6]. Evaluating AI behavior within such environments requires metrics that reflect real-world development conditions rather than isolated, greenfield coding tasks.

A. Definitions

Language Learning Model (LLM): A large language model is a neural network trained on massive text datasets to understand, generate, and predict human language.

Meaningful Session: A development session with Claude that includes substantial code changes, dismissing exploratory sessions (intensive interaction with Claude but minimal/zero code changes) but including sessions with testing, refactoring, or new functionality.

Reprompt: A single additional prompt or clarification required after Claude's initial output fails to meet task specifications or produces errors.

IV. Methodology

Our methodology consists of three phases: defining evaluation metrics, selecting tasks within the existing application, and characterizing Claude's performance across multiple development sessions.

A. Defining Metrics

We define the following measurable criteria for AI performance:

- **Mutation Test Accuracy:** Percentage of mutation tests passed before and after AI modifications.
- **Regression Count:** Number of existing tests broken by AI-generated code.
- **Reviewer Quality Scores:** Human-assessed readability, maintainability, and logic clarity (1–10 scale).
- **Code Quality Analysis:** Linter outputs, static-analysis warnings, and presence of issues such as code duplication or missing type hints.
- **Developer Intervention Time:** Time spent debugging or correcting Claude's output.
- **Iteration Count:** Prompt rounds required before a task reaches completion.

TABLE I
TOKEN USAGE

Category	Value
Total Token Budget (per session)	200,000
Total Tokens Used (all passes)	311,426
Remaining Tokens (latest session)	88,574
Latest Session Usage (%)	55.71%
Overall Usage (%)	>100%

TABLE II
TESTING OUTCOMES ACROSS DEVELOPMENT PASSES

Pass	Res.	Fail	Err	Skip
1	F	15+	–	–
1 (report)	F	–	23	3
2	F	3	2	3
3	F	1	2	3
4	F	1	2	3
5	F	1	1	3
6	P	0	0	–
7	P	0	0	–
8	F	6	5	3
9	F	6	5	3
10	F	6	5	3
11	P	0	0	–

B. Task Selection

We evaluate Claude on tasks that involve:

1) Modifying existing files
2) Extending application features
3) Updating configuration logic
4) Adjusting test suites and fixing failing tests
5) Adding standalone new modules compatible with legacy patterns

C. Development Characterization

For each task, we record:

- Claude's initial output
- Test results after changes
- Reviewer notes
- Recurring failure patterns
- Number of corrections required

With these metrics we hope to provide a more comprehensive view than many other teams that came before us. To provide an understanding of AI code creation that may be measurable and transferrable to provide a guideline for future research in this field.

V. RESULTS

The data we were able to grab varied by time and location, Here the results we were able to gather from our research using Claude code are as follows.

A. Test case Feature: Scoring Transparency

In this case we were able to collect data while writing, designing, and fixing the Scoring transparency feature.

From the above section we can extrapolate that Claude usage in feature creation especially on medium sized projects can be intensive in both token usage and the amount of passes required to produce production quality code.

VI. CASE STUDY

A. Case Study 1: Testing Instability

Claude frequently introduces regression failures when modifying existing logic. Although feature implementation is fast, stability is inconsistent, which requires significant developer verification. Additionally, extensive sessions result in the side effect of failing to generate test cases for new code, possibly due to forgetting operating guidance with an increasing mass of tokens.

The most prominent evidence of Claude's difficulty with testing comes from our First-Run failure statistics, defined by the first run of tests post-modification of program code. Our classification divides the failures into two categories; by session and by run. First run failures by session are classified by a session containing at least one first run failure, which means that a session can have any number of first run failures from 0% to 100% of all test runs.

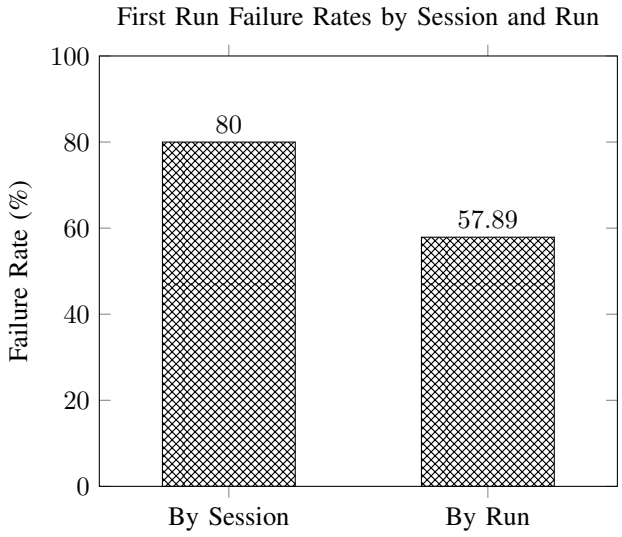


Fig. 1. First Runs: First Execution of Tests Post-Modification.

We also wanted to examine Claude's ability to work through troubleshooting failed tests. To do this we collected sequences of test runs on a statics subset of tests, usually after new tests were written. This data was collected both manually, and by analyzing local Claude Code session logs for sequences of invocations of Django's test function. From the logs we were able to extract exact details of the test commands run, the output to determine number of tests failed each run, and the timestamps, which allowed us to calculate the statistics in III.

We found that while Claude certainly has a hard time getting everything right the first time, generally it gets much more correct than wrong, only about 10% of tests failed the first time and, on average, Claude was able to make corrections and achieve all tests passing on about the 3 run of the tests and only required on average 9 minutes. The time is somewhat meaningless, scientifically, as there was significant variation in the number of tests run each sequence, and likely much further variation in the actual execution time of each individual test,

TABLE III
ANALYSIS OF TEST TROUBLESHOOTING

Category	Value
Test Sequences	21
Total Tests	487
Average Initial Run Test Failure Rate(%)	10.63%
Average Test Runs to Debug all Failures	3.14
Average Time to Debug	8.52 minutes

as well as if there were pauses in the process due to delayed user commands. But it does demonstrate that Claude was able understand the root cause of test failures to make corrections to failing code very quickly.

B. Case Study 2: Legacy Code Friction

Claude performs significantly better with fresh files than when navigating older code with deeply interconnected components. It struggles to infer undocumented patterns, causing repeated rewrite loops. Re-prompting after a response modified important code or caused conflicts with key features was common.

Metric	Value
Meaningful Sessions	99
Avg. Reprompts Per Session	93.88
Total Reprompts	9294

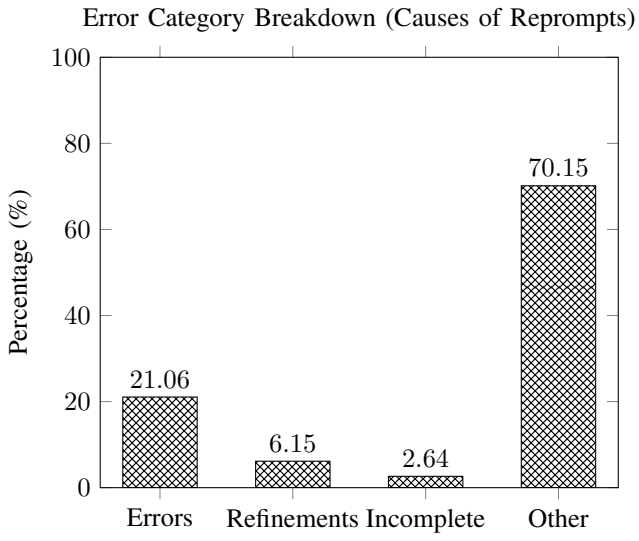


Fig. 2. Reprompt Metrics and Error Category Breakdown. ('Other' May Dominate Due to Collaborative Workflow and/or Categorization Methods)

C. Case Study 3: Configuration and Tooling Errors

Claude occasionally references tools, variables, or configurations that do not exist in the project. These errors require manual research and cleanup.

D. Results

In this section, we report quantitative metrics from the AI-assisted development process, including token usage, testing outcomes across passes, and an overall session-level summary.

1) Code Changes: Between the undergrad class project and the current main branch over 131,000 lines of new code were added with minimal deletions. These metrics represent lines added and removed thus there is some bias in the data since changing a line and adding a new one show up in the same way. However, this still reflects how Claude and other LLM coding system bias creation rather than refactoring or reuse.

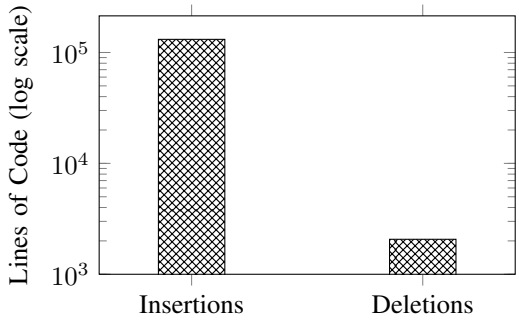


Fig. 3. Log-scaled distribution of source code insertions and deletions between the last commit of the previous repo `d95a2ac` and the current main branch.

2) Mutation Testing: Mutation testing was conducted using MutMut version 2.4.0 to evaluate the effectiveness of the test suite after all AI-generated features and code repair. A total of 2,563 mutants were generated and evaluated following an initial baseline test run to establish execution time thresholds. Of these mutants, 416 were successfully killed by the test suite, while 2,066 triggered timeouts due to excessive execution time relative to the baseline. An additional 81 mutants were classified as suspicious due to elevated execution times, and no mutants survived or were skipped. According to Claude 65% timeout rate is acceptable for a program that has Open AI integrations, however ours here is 80.61%, furthermore our suspicious count was only 3.16% and our tests that survived were 0% which is good [7]. Per Claude our timeout time was 26 seconds with a 2.6 time multiplier which is normal for mutation testing which according to Claude is normal [7]. However running all of our unit tests does normally take about 20 minutes to complete. For 100% test coverage running mutation tests took around 20 hours with a shorter timeout on an I9 processor. Though this did give us an effectiveness rate or mutation score of 16.23% via this formula from chat-GPT Effective detection rate = Killed / Total [8]. For reference the original was about 22% which is better than ours likely due to the large amount of timeouts we had in our run.

Mutation Outcome	Count
Full Test Suite Run	
Timeout	26s
Killed Mutants	416
Timed Out	2,066
Suspicious	81
Survived	0
Skipped	0
Total Mutants	2,563
Focused Module Run	
Timeout	200s
Killed Mutants	87
Timed Out	0
Suspicious	0
Survived	283
Skipped	0
Total Mutants	370

TABLE IV
MUTATION TESTING RESULTS OBTAINED USING MUTMUT. THE FIRST BLOCK REPORTS RESULTS FROM THE FULL TEST SUITE ACROSS THE ENTIRE APPLICATION, WHILE THE SECOND BLOCK REPORTS RESULTS FROM A FOCUSED MUTATION RUN TARGETING CORE APPLICATION MODULES.

On the Second run our professor was kind enough to run mutation tests on an AWS server for maximum processing running 6 agents at a time on a que of modules he was able to return the following results.

TABLE V
MUTATION TESTING CAMPAIGN SUMMARY

Metric	Value
Total Mutations Generated	1,376
Killed Mutants	629
Survived Mutants	747
Modules Tested	15
Modules Skipped	6
Execution Time	∼47 minutes
Parallel Workers	6
Overall Mutation Score	45.7%

For the mutation score on this one it's slightly disappointing though still did better than the previous repository. We can see at 75% of testing coverage we still get a score of 44% which is almost double of the 22% we got when we first started working on this repository. The testing coverage multiplied 3.6 times what it was to 1,333 from 370 before or our of the original 2563 it's 6.9 times the tests.

As for what modules did the best we can see that the rate_limit_config then 79% with job_listing_parser and all the way down to 0% in token_tracking and invitation_utils. I suspect that this has something to do with how many times we had to refactor the tests before they passed, and how many times they were refactored after implementing new features, along with the redundant and overlapping coverage that Claude produced with each feature. Despite all of this after consulting with Chat-GPT 40-60% is still a strong score as far as mutation scores go, especially for projects of our size [8].

3) Test Case Feature: Scoring Transparency:

VII. DISCUSSION

Claude Sonnet 4.5 demonstrates strong capabilities in rapid development, CI/CD integration, documentation generation, and implementing secure coding practices. However, weaknesses appear in:

- Maintaining test stability
- Interpreting complex legacy patterns
- Understanding environment-specific configurations
- Performing linting corrections
- Avoiding repeated change–revert cycles

When it comes to maintaining test stability and performing linting corrections, Claude struggles to keep tests running, especially if they are tests it wrote. Claude has a hight likelihood of producing shallow tests to meet coverage metrics. Claude will then often break those tests to conform to linting standards. Human attention is still needed when it comes to these metrics.

Claude will also have a hard time understanding legacy patterns and environment specific configurations. It currently cannot fully grasp and application's use and it's environment without extensive documentation or explanation from the developer running Claude. Once again the responsibility of integration and stability falls to human minds to ensure Claude is improving the legacy code in a safe way.

However looking back at the repository the mutation score doubled and overall implements a lot more features with intended functionality and in some cases more than the stories accounted for. As for testing even-though it wrote a plethora of redundant tests it still did maintain ¿80% test coverage for the entire repository, albeit with a lot of re-prompting and sometimes manual direction. As far as mutation scores go it did actually do a decent job, we believe this could be a lot better and much more efficient. However it did double the score. The detraction from that is that the origional tests weren't professionally written, and the sheer number of unit tests likely drove up the average.

Overall these findings indicate that AI tools require human oversight and verification when working within non-trivial and pre-existing applications.

VIII. CONCLUSION

This study establishes a measurable framework for evaluating AI performance in the often overlooked domain of legacy software maintenance. Our findings suggest that while Claude Sonnet 4.5 is a powerful tool for development tasks, it cannot yet replace the architectural oversight of a human engineer.

Claude model demonstrated a capability to navigate undocumented code but struggled to maintain system stability without causing regressions. The high levels of test failures and the necessity for extensive re-prompting indicate that AI tools are currently best utilized as "Junior Developers" with high output but requiring review and a significant amount of oversight.

For the industry, this implies that integrating AI into legacy pipelines requires a strong preexisting set of automated testing to prevent regressions that the model is prone to generating. As AI models evolve, future success in legacy environments will likely depend less on raw coding speed and more on the model's ability to maintain context over long operational windows and prioritize system stability over feature velocity.

TABLE VI
PER-MODULE MUTATION TESTING RESULTS

#	Module	Status	Total	Killed	Survived	Score (%)	Duration
1	ratelimit_config	COMPLETED	27	27	0	100.0	5.1 min
2	permissions	COMPLETE	43	39	4	90.7	5.0 min
3	job_listing_parser	COMPLETE	162	129	33	79.6	19.3 min
4	bias_detection	COMPLETE	191	132	59	69.1	40.7 min
5	utils	COMPLETE	8	5	3	62.5	0.8 min
6	openai_utils	COMPLETE	81	38	43	46.9	19.0 min
7	resume_parser	COMPLETE	101	46	55	45.5	19.2 min
8	serializers	COMPLETE	226	101	125	44.7	47.1 min
9	audit_utils	COMPLETE	19	8	11	42.1	7.7 min
10	forms	COMPLETE	311	100	211	32.2	86.7 min
11	constants	COMPLETE	47	3	44	6.4	19.9 min
12	signals	COMPLETE	35	1	34	2.9	4.3 min
13	token_tracking	COMPLETE	33	0	33	0.0	20.1 min
14	invitation_utils	COMPLETE	92	0	92	0.0	41.4 min
15	pdf_export	SKIPPED	–	–	–	–	Baseline failed
16	user_data_utils	SKIPPED	–	–	–	–	Baseline failed
17	latency_utils	SKIPPED	–	–	–	–	Baseline failed
18	model_tier_manager	SKIPPED	–	–	–	–	Baseline failed
19	models	SKIPPED	–	–	–	–	Baseline failed
20	report_utils	SKIPPED	–	–	–	–	Baseline failed
21	views	SKIPPED	–	–	–	–	Too large

IX. FUTURE WORK

While this project demonstrates the efficacy of using a single LLM agent (Claude) for legacy code development within a semester-long time-frame, several promising directions emerge for extending and enhancing this approach.

A. Multi-Agent Architecture Exploration

Our single-agent methodology could be enhanced by incorporating elements from multi-agent systems research [2], [3]. Future work could investigate hybrid architectures where a primary agent coordinates with specialized sub-agents for distinct tasks such as code review, testing, and documentation generation. This would maintain the contextual continuity advantages of our single-agent approach while leveraging the benefits of role specialization. Such an architecture could address scenarios where different aspects of legacy code require distinct expertise while maintaining a unified understanding of the code base.

B. Enhanced Context Management

A significant limitation in extended legacy code projects is context window management as code bases grow. Future iterations could explore techniques for intelligent context pruning, where the agent selects relevant code sections based on the current task while maintaining awareness of broader system architecture. Additionally, developing persistent memory mechanisms that allow the agent to recall previous interactions and decisions across multiple development sessions would improve continuity in longer projects. Exploring conversation focused development rather than 'invocation' could also be a viable means of developing with agents and legacy code [9].

C. Metrics and Evaluation Framework

Establishing metrics for evaluating legacy code development remains an open challenge. Future work could develop comprehensive evaluation frameworks that measure not only code quality (correctness, maintainability, performance) along side process efficiency (time to understand legacy code, number of iterations required for successful refactoring). Comparative studies between our single-agent approach and multi-agent frameworks could identify specific scenarios where each architecture excels, providing practical guidance for tool selection in legacy modernization. There also is substantial progress to be made in our current metrics

D. Domain-Specific Specialization

The general-purpose nature of Claude could be enhanced through domain-specific fine-tuning or prompting strategies tailored to particular legacy technology stacks. Future research could investigate how to effectively prime the agent with specialized knowledge about obsolete programming languages, deprecated frameworks, or domain-specific architectural patterns common in legacy systems. This would address the expertise gap noted in current LLM research while maintaining the single-agent architecture's simplicity and contextual coherence. However, the consideration of the effectiveness of such a strategy should be compared to a fresh agent trained with only the domain-specific specialization in mind to scale the efficacy of both.

E. Scaling to Larger Code bases

Our project operated on a moderately sized legacy code base. Future work should investigate scalability to enterprise level systems with millions of lines of code, complex interdependencies, and multiple subsystems. This may require developing a hierarchy for code comprehension as considered above with multi-agent architecture.

REFERENCES

[1] J. Yang, C. Jimenez, A. Wettig, K. Lieret, S. Gupta, O. Press, K. Huang, H. Jain, J. Yang, Y. Agarwal, R. Krishna, and D. Zhou, "Agent-computer interfaces enable automated software engineering," in

Advances in Neural Information Processing Systems (NeurIPS), 2024. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/file/5a7c947568c1b1328ccc5230172e1e7c-Paper-Conference.pdf

[2] J. He, C. Treude, and D. Lo, "Llm-based multi-agent systems for software engineering: Literature review, vision, and the road ahead," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 5, May 2025. [Online]. Available: https://doi.org/10.1145/3712003

[3] K. Cinkusz, J. A. Chudziak, and E. Niewiadomska-Szynkiewicz, "Cognitive agents powered by large language models for agile software project management," *Electronics*, vol. 14, p. 87, 2024. [Online]. Available: https://arxiv.org/pdf/2508.16678

[4] D. Tosi, "Studying the quality of source code generated by different ai generative engines: an empirical evaluation," *Future Internet*, vol. 16, p. 188, 2024. [Online]. Available: https://www.mdpi.com/1999-5903/16/6/188

[5] M. Taeb, H. Chi, and S. Bernadin, "Assessing the effectiveness and security implications of ai code generators," *Journal of the Colloquium for Information Systems Security Education*, vol. 11, p. 6, 2024. [Online]. Available: https://cisse.info/journal/index.php/cisse/article/view/180/180

[6] Y. Ramaswamy, "Technical debt and devops: Strategies for managing legacy systems in a ci/cd world," *NeuroQuantology*, vol. 21, no. 7, pp. 1256–1267, 2023.

[7] Anthropic, "System card: Claude opus 4 & claude sonnet 4," [Online]. Available: https://www.anthropic.com/claude-4-system-card, May 2025, accessed: 2025-12-14.

[8] OpenAI, "Gpt-5 large language model (chatgpt interface)," [Online]. Available: https://chat.openai.com/, 2025, used for code reasoning, debugging assistance, and manuscript editing. Accessed: 2025-12-14.

[9] S. I. Ross, F. Martinez, S. Houde, M. Muller, and J. D. Weisz, "The programmer's assistant: Conversational interaction with a large language model for software development," in *Proceedings of the 28th International Conference on Intelligent User Interfaces*, ser. IUI '23. ACM, Mar. 2023, p. 491–514. [Online]. Available: http://dx.doi.org/10.1145/3581641.3584037